

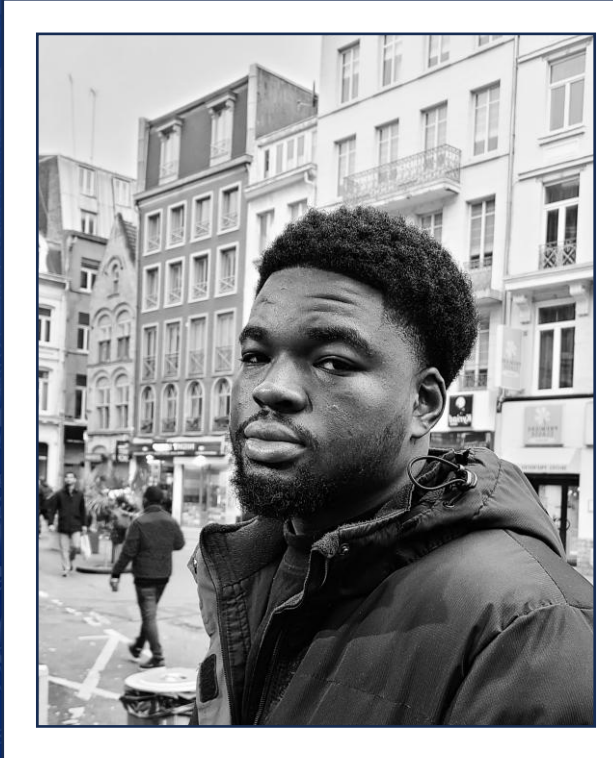


EuroScipy 2026

**Lessons from Building a Large-
Scale Engineering Simulation Data
Processing Library**

21st of July

SPEAKER



Muhammed Adeyemi, ADEDIGBA
Research and Development Engineer – PyAnsys Core Team
Ansys (part of Synopsys)
BEng. in Civil Engineering, Msc. in Advanced Solid Mechanics
I like football, video games, and light calisthenics

Agenda

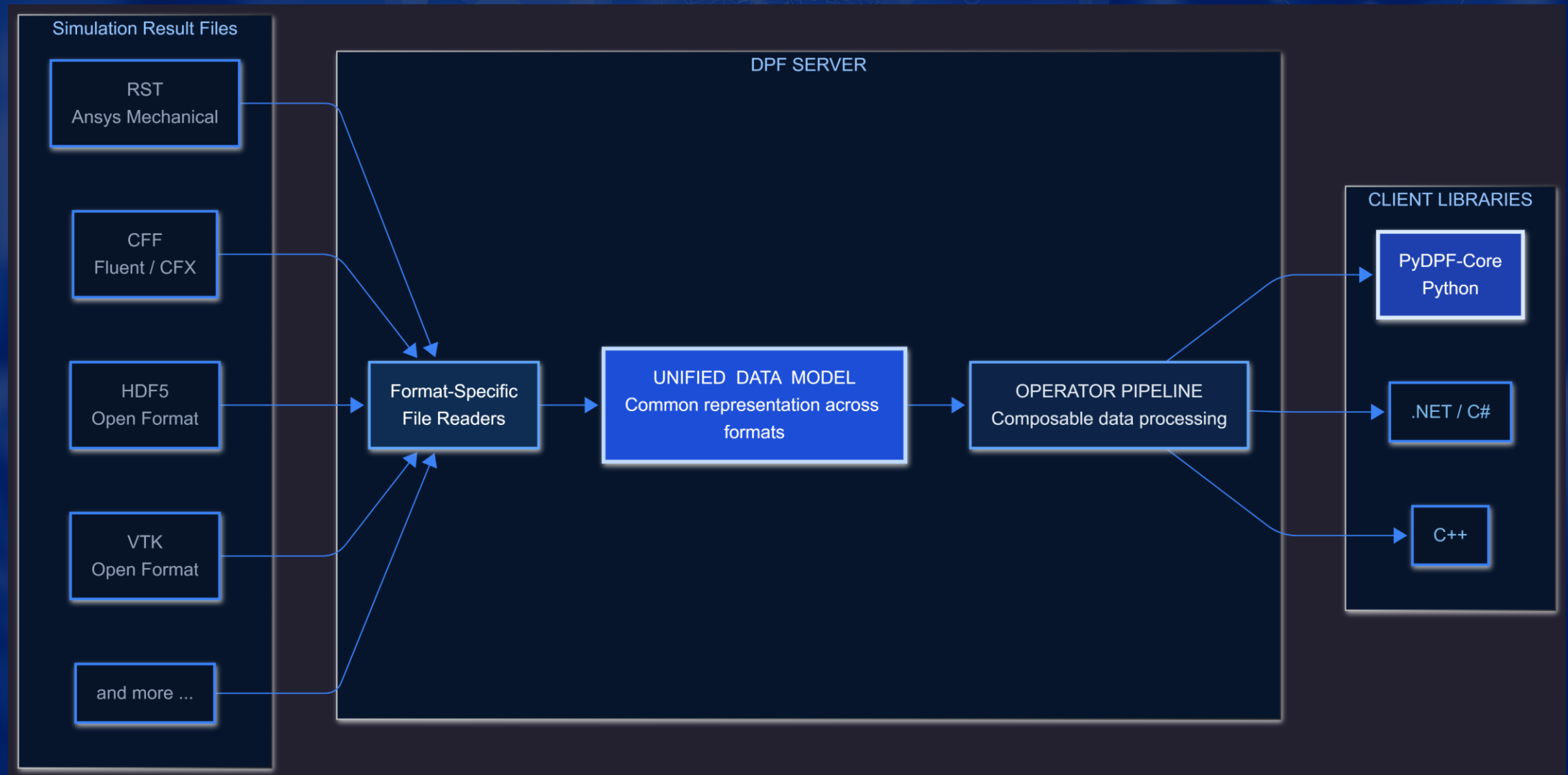
- ⌘ **PyDPF-Core (some context)**
- ⌘ **Challenges we face / faced**
- ⌘ **How we have addressed some of them (lessons)**
- ⌘ **Q and A**

Context

- 🌀 **PyDPF-Core is the Python client library for Ansys Data Processing Framework (DPF)**
- 🌀 **DPF is a performant physics-agnostic framework for pre/post-processing of simulation data.**
- 🌀 **DPF is:**
 - **highly customizable**
 - **extensible**
 - **supports remote/distributed access**



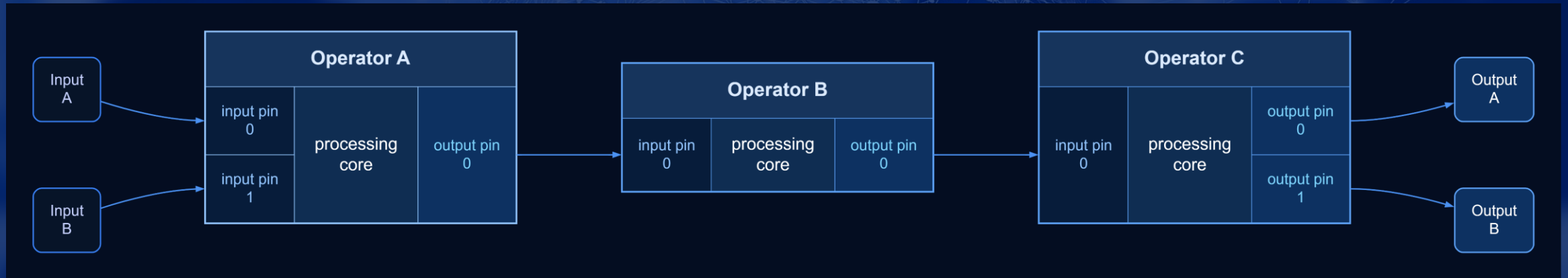
Context



The Data Processing Framework

Context

- ⌘ The core philosophy of DPF is to compose complex data processing pipelines/workflows from simpler computation units known as operators



Chained operators for transforming simulation data

- ⌘ Another good example of where this philosophy exists is VTK's (Visualization Toolkit) algorithms.

The background is a dark blue gradient. In the center is a large, glowing sphere composed of a complex network of white lines and dots, resembling a data network or blockchain structure. Surrounding this central sphere are several semi-transparent hexagonal icons. These icons include: a target-like symbol with concentric circles and a central dot; a line graph with four data points; a bar chart with four vertical bars of increasing height; a Bitcoin logo; a circular diagram with eight segments; and a circular diagram with three overlapping circles. The overall aesthetic is technological and digital.

CHALLENGES

Challenges

⌘ Multiplicity of data representation / source



.format_A



.format_B



.format_C



```
1 match file_format:  
2     case "format_A":  
3         #do something  
4     case "format_B":  
5         #do something else  
6     case "format_C":  
7         #do some other thing
```

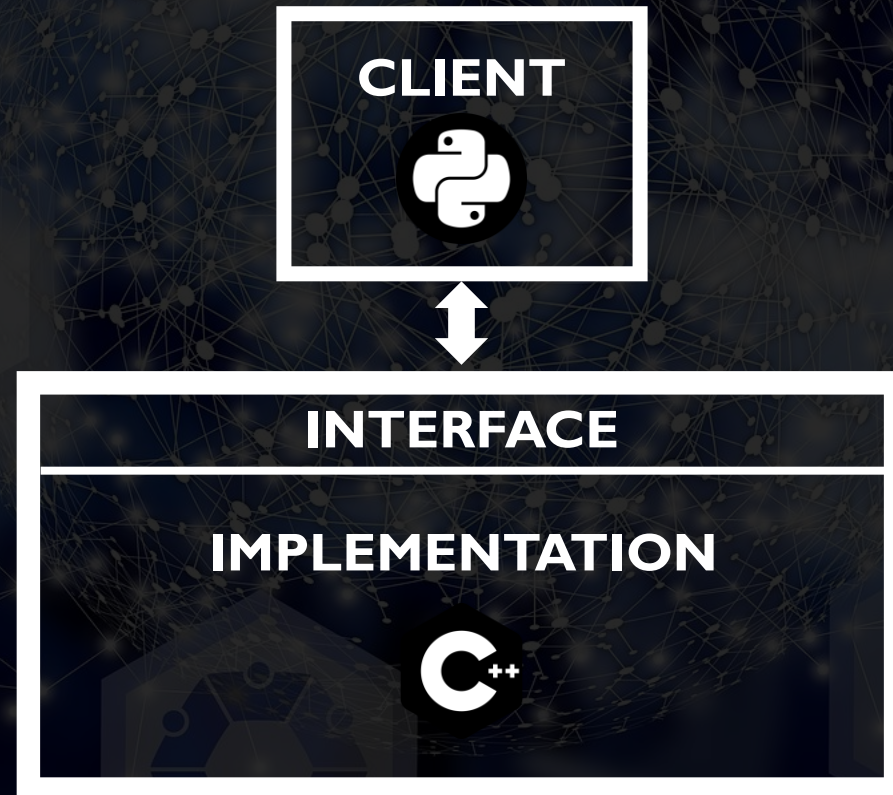
You may need to handle
different representations
of data to be processed by
your library



And you should strive to
avoid this by all means

Challenges

- ⌘ Decoupling the evolution of client(s) from the backend / server



Challenges

- ⌘ **Decoupling the evolution of client(s) from the backend / server**
- ⌘ **Exposing your client API in multiple languages**



Challenges

- 🌀 Ensuring composability, flexibility, and maintainability



Challenges

- ⌘ Improving performance, minimizing expensive computations, and reducing data transfers
- ⌘ Managing growing server APIs on the client side
- ⌘ Visualizing processed data
- ⌘ Ensuring up-to-date documentation, sustainable CI/CD practices, etc.

The background is a dark blue gradient. In the center is a large, glowing sphere composed of a complex network of white lines and dots, resembling a molecular or digital structure. Surrounding this central sphere are several semi-transparent hexagonal icons. These icons include: a target or radar-like symbol at the top left; a line graph with an upward trend at the top right; a bar chart with four bars of increasing height on the middle left; a Bitcoin logo on the middle right; a bar chart with four bars of decreasing height at the bottom left; and a circular diagram with eight segments at the bottom right. The overall aesthetic is high-tech and digital.

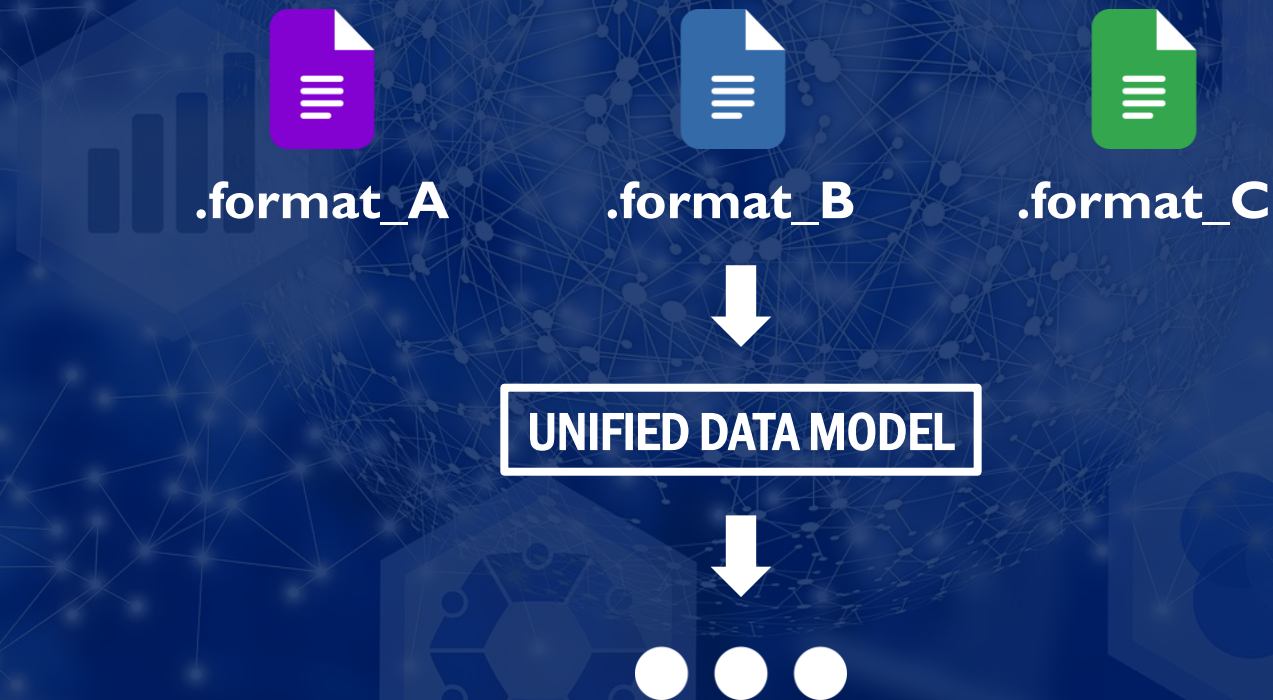
LESSONS

Lessons

🌀 **Challenge: Multiplicity of data representation / source**

🌀 **How to address?**

- **A good abstraction is the key**
- **And that is achievable via a generalized data model**



Lessons

- ⌘ **Challenge: Multiplicity of data representation / source**
- ⌘ **How to address?**
 - **A good abstraction is the key**
 - **And that is achievable via a generalized data model**
- ⌘ **As an example, in DPF, a “Field” is the abstraction used. It is a data model that contains:**
 - **The actual simulation data**
 - **Spatial information**
 - **Temporal information**
 - **Other metadata**

Thus, making it a self-describing data model.

Lessons

- ⌘ **Challenge: Decoupling the evolution of client(s) from the backend / server and exposing your API in multiple client languages**

Lessons

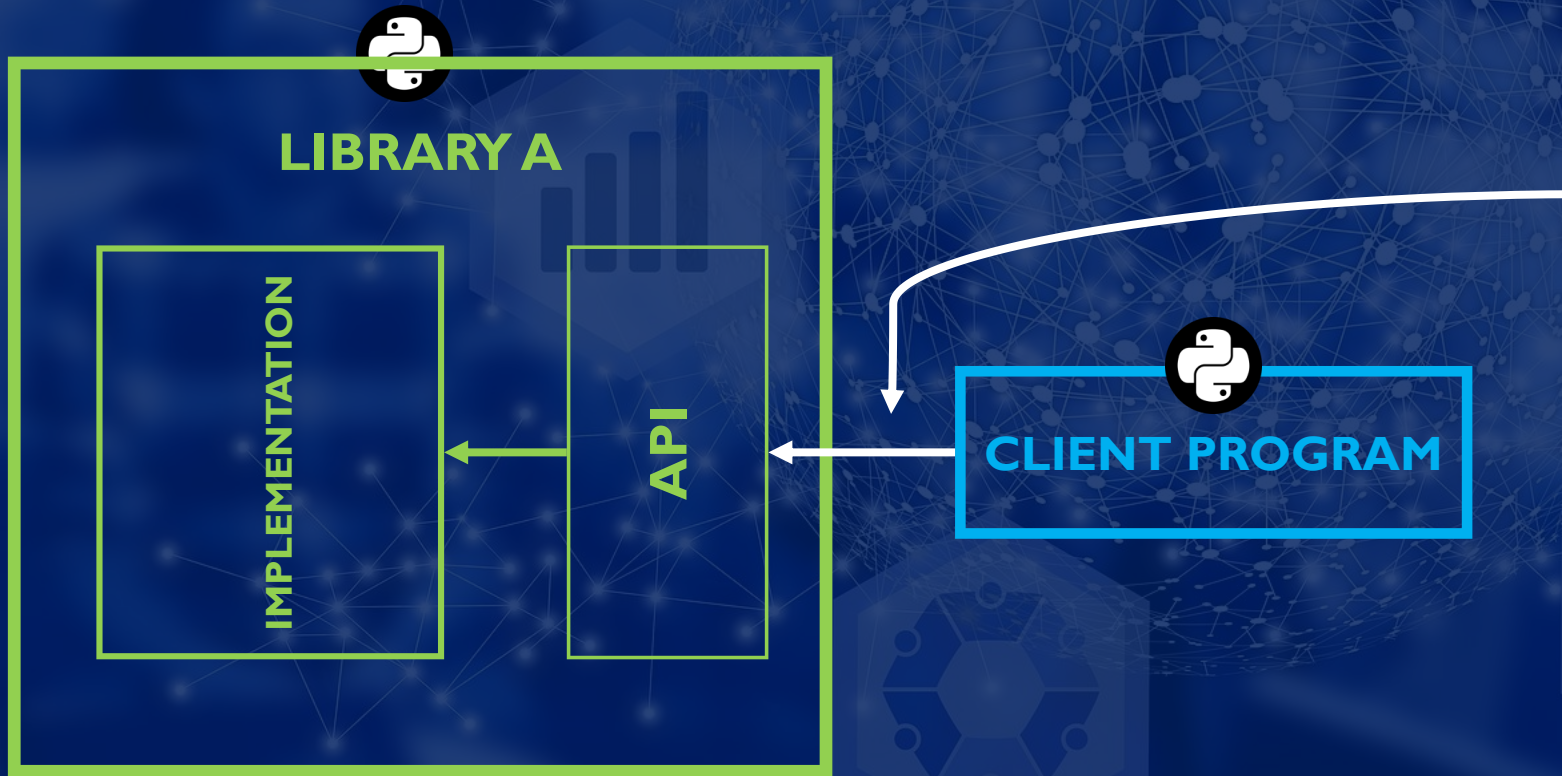
- ⌘ **Challenge: Decoupling the evolution of client(s) from the backend / server and exposing your API in multiple client languages**
- ⌘ **Let us consider this challenge in a bit more detail.**

Lessons

- ⌘ Let us consider this challenge in a bit more detail.
- ⌘ When developing in a compiled language like **C++**, maintaining a stable **ABI** is very important.
- ⌘ And keeping the **ABI** stable can lead to strong coupling.
- ⌘ But what is the **ABI** (Application Binary Interface)?

Lessons

- ⌘ But what is the **ABI** (Application Binary Interface)?
- ⌘ It is fairly complex to define. Let us draw an analogy with the **API**.
- ⌘ Consider what happens in an interpreted language like **Python**:

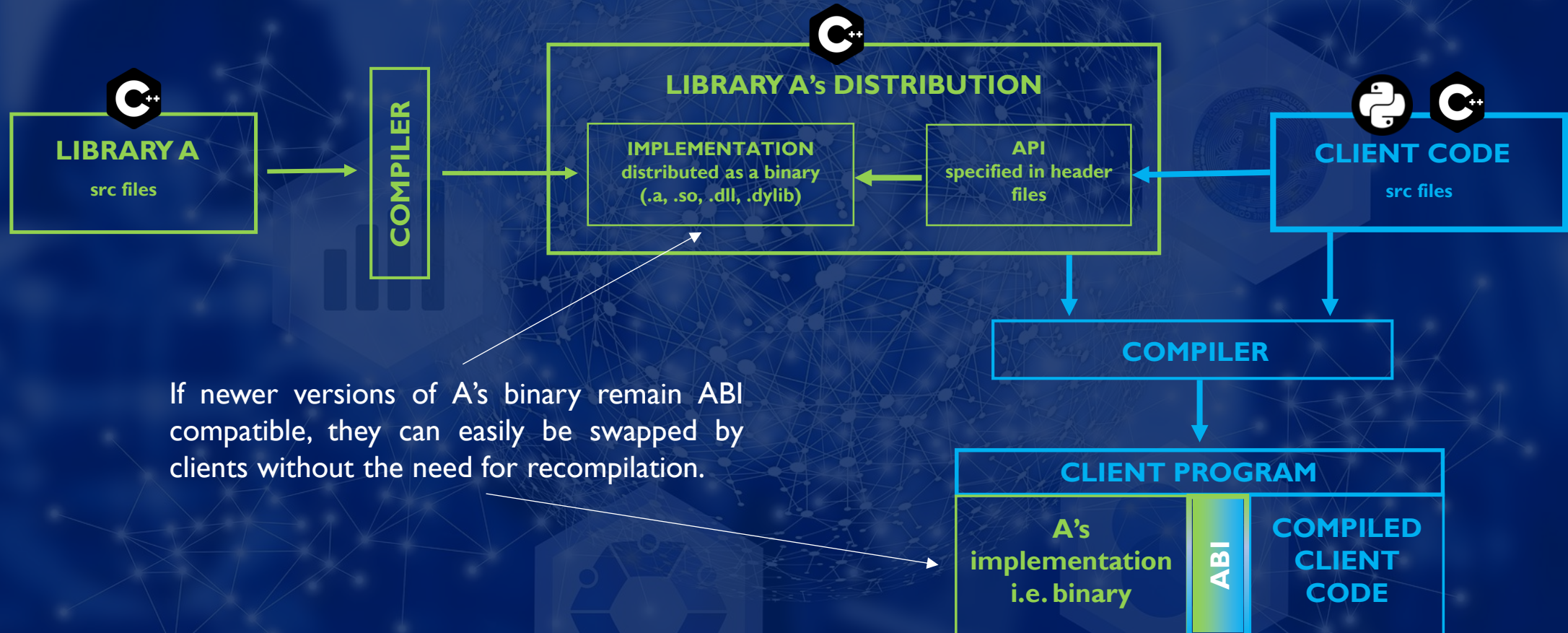


As long as the subset of A's API used in the client code doesn't change, the client program will not be impacted by the following changes in A:

- A's implementation
- A's tooling
- A's dependencies
- Other subset of A's API

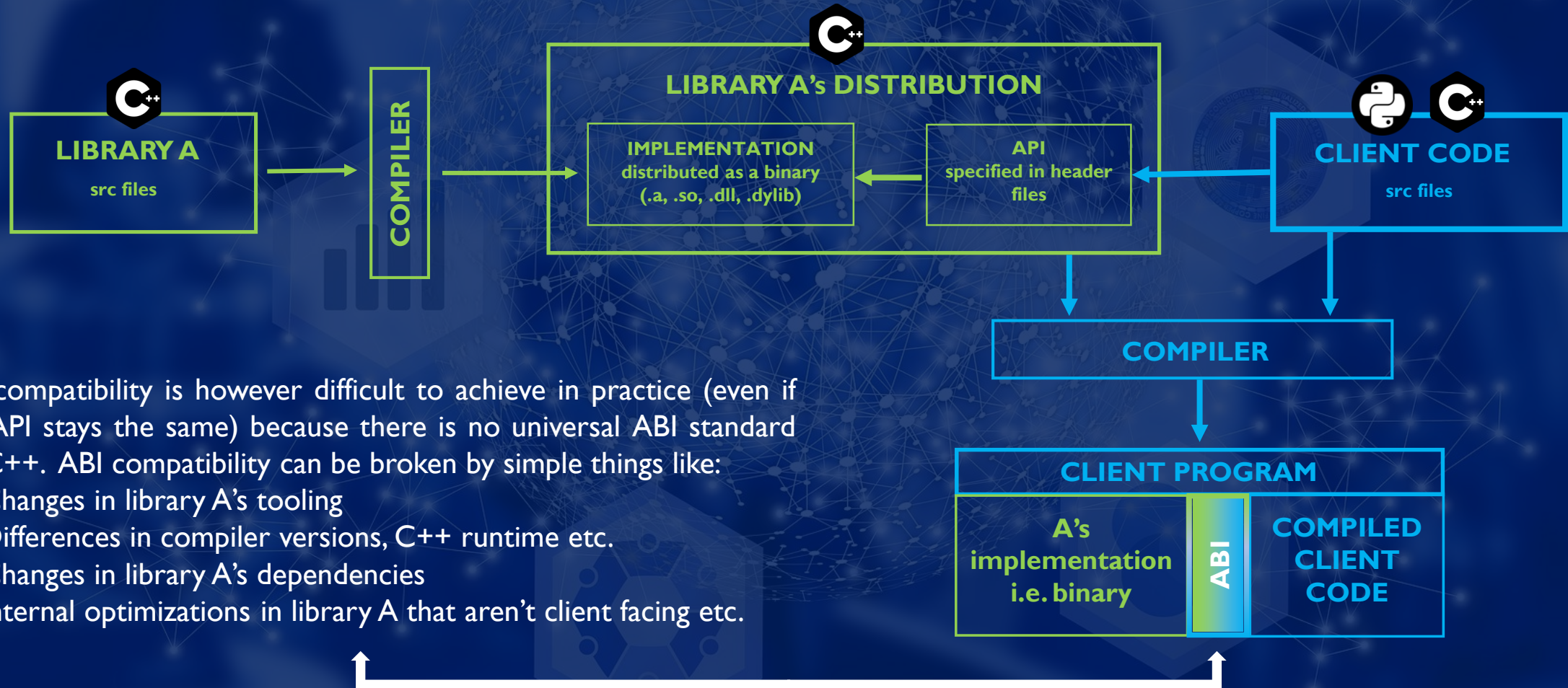
Lessons

- For software written in compiled languages like C++, interface compatibility must also be maintained between binaries.



Lessons

- For software written in compiled languages like C++, interface compatibility must also be maintained between binaries.

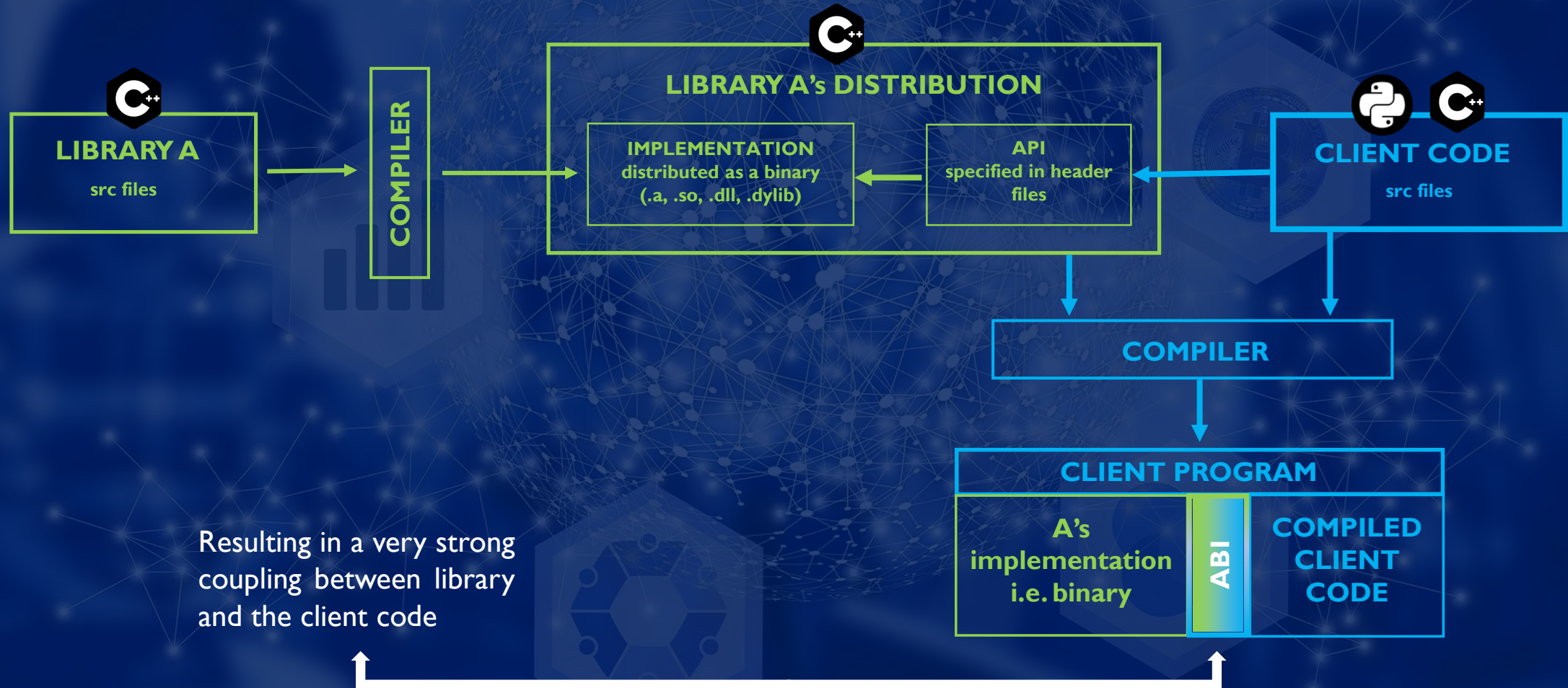


ABI compatibility is however difficult to achieve in practice (even if the API stays the same) because there is no universal ABI standard for C++. ABI compatibility can be broken by simple things like:

- Changes in library A's tooling
- Differences in compiler versions, C++ runtime etc.
- Changes in library A's dependencies
- Internal optimizations in library A that aren't client facing etc.

Lessons

- For software written in compiled languages like C++, interface compatibility must also be maintained between binaries.



Lessons

- ⌘ For software written in compiled languages like C++, interface compatibility must also be maintained between binaries.
- ⌘ In summary, for compiled languages, “interface = API + ABI”. Maintaining a stable ABI can prevent decoupling if not handled properly.

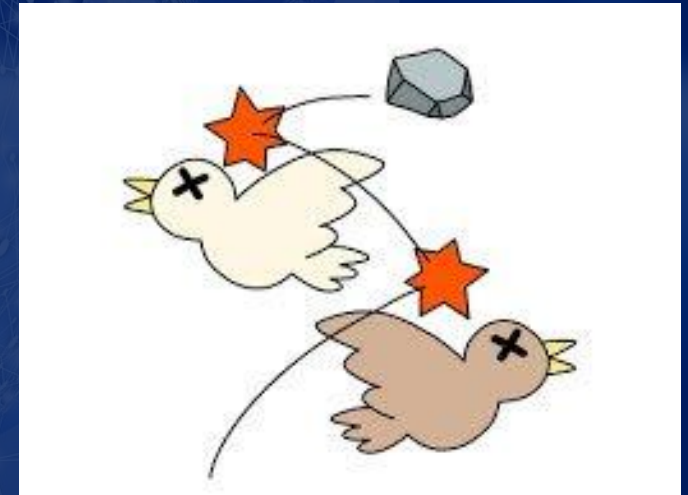
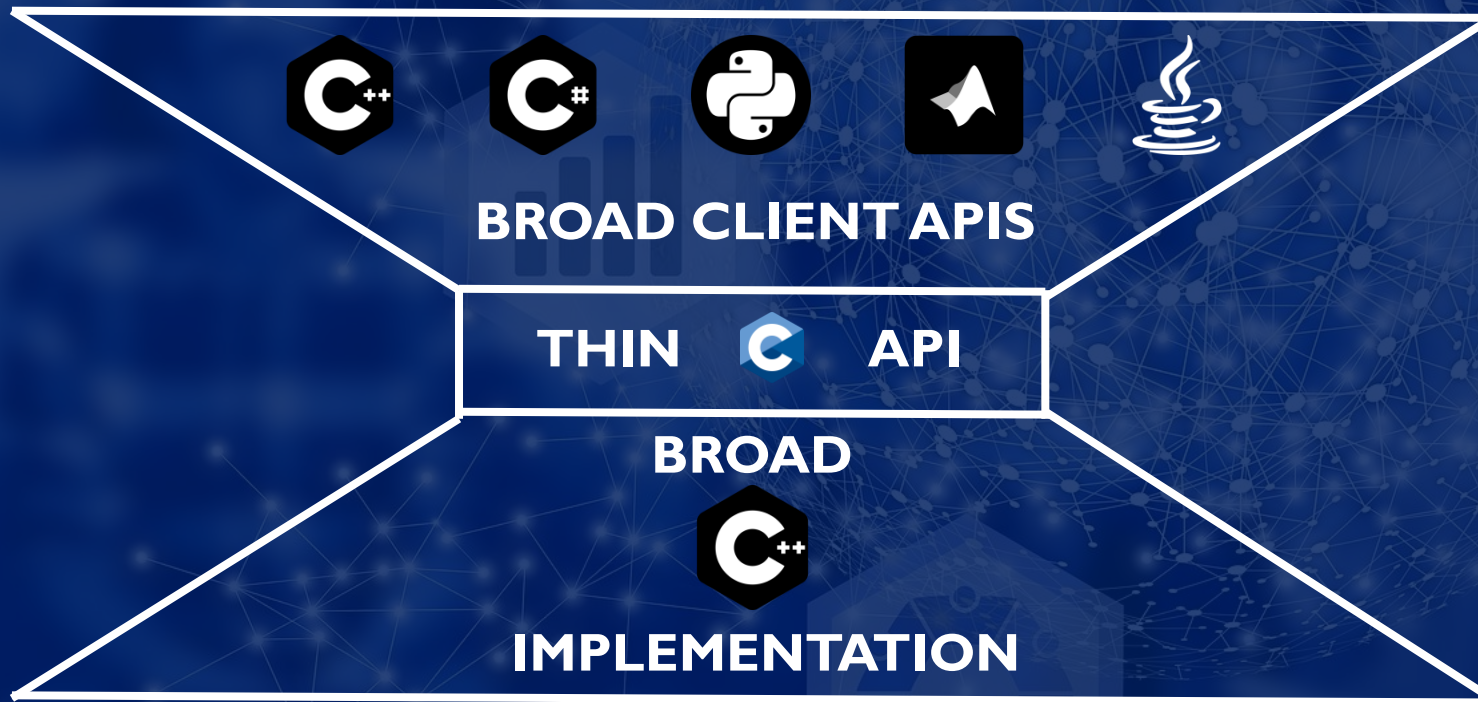
Lessons

- ⌘ **Challenge: Decoupling the evolution of client(s) from the backend / server and exposing your API in multiple client languages**
- ⌘ **How to address?**
- ⌘ **Enters the Hourglass Interface Pattern**



Lessons

- 🌀 The Hourglass Interface Pattern
- 🌀 You kill two birds with one stone.



Lessons

- 🌀 **The Hourglass Interface Pattern**
- 🌀 **You kill two birds with one stone.**
- 🌀 **How does the pattern help?**
 - **ABI stability problem / Decoupling → C has a very stable and almost universal ABI**
 - **Client API in multiple languages → Almost all programming languages have FFI mechanisms for calling C functions:**
 - **ctypes and cffi for Python.**
 - **JNI / JNA for Java**
 - **P / Invoke for C# / .NET**
 - **Etc.**
- 🌀 **The duration of this talk is not enough to do justice to the technical details involved. I invite the audience to search “CppCon talk on Hourglass Interfaces for C++ APIs by Stefanus Du Toit”.**

Lessons

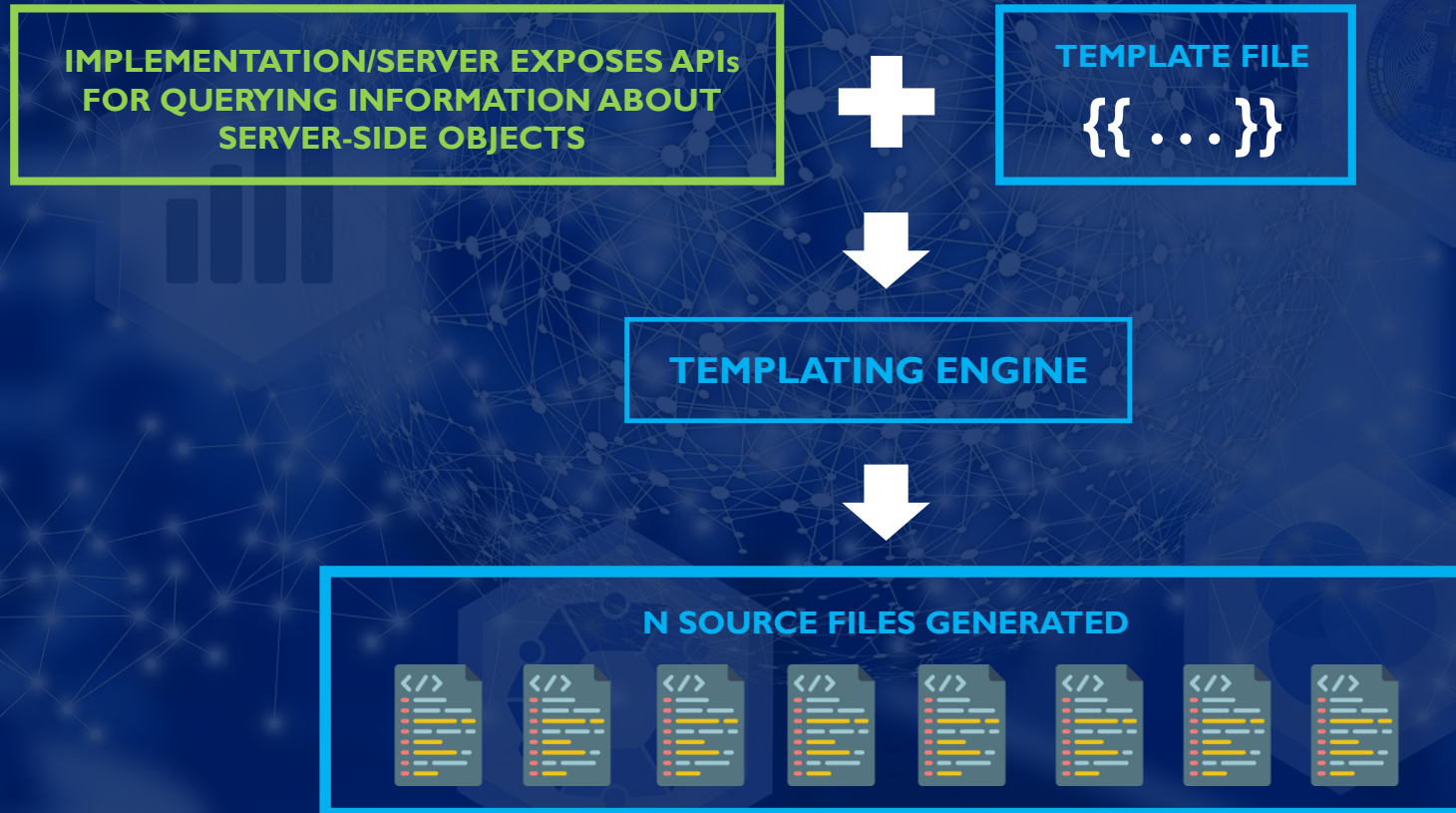
⌘ **Challenge: Improving performance, minimizing expensive computations etc.**

⌘ **How to address?**

- **Lazy evaluations**
- **Caching**

Lessons

- 🌀 **Challenge: Managing growing server APIs on the client side**
- 🌀 **How to address?**
- 🌀 **Several options available in Python: Jinja, Moustache, Cookiecutter etc.**



Lessons

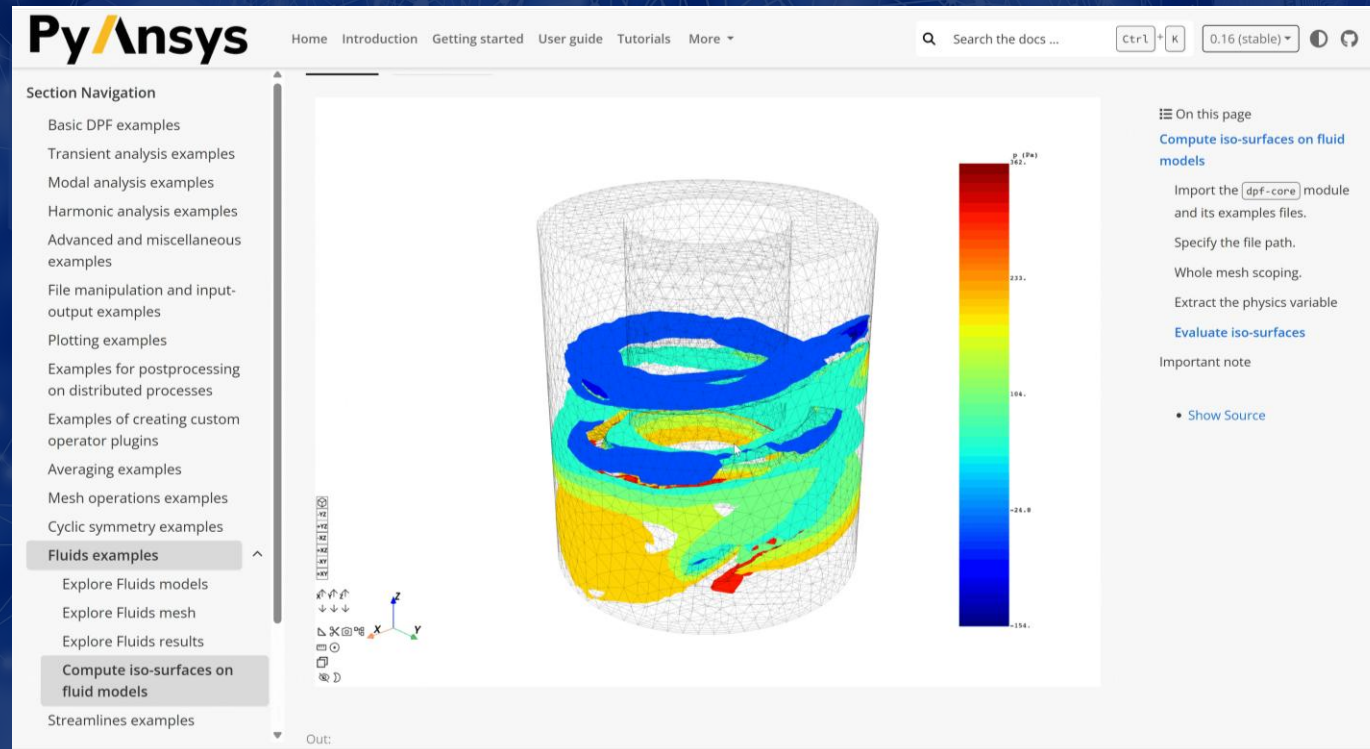


Challenge: Visualizing the processed data



How to address?

- **For 3D visualization, our go-to is the Visualization Toolkit (more specifically PyVista). Other libraries like vedo can also work well depending on your use case.**
- **Numerous options are available for 2D visualization e.g. Matplotlib, Seaborn, Plotly etc.**



Lessons

- 🌀 **Challenge: Ensuring up-to-date documentation, sustainable CI/CD practices, etc.**
- 🌀 **How to address:**
 - **For documentation, a good recommendation is the Sphinx ecosystem. It is very rich and matured; and CI can help automate almost every all of documentation generation.**
 - **As a minimum, have CI/CD pipelines for testing, generating documentation, vulnerability checks, and releasing.**

Lessons

⌘ **Challenge: Ensuring composability, flexibility, and maintainability**

⌘ **How to address:**

- **By doing a bit of everything already discussed.**
- **Keep things simple, even if it means writing more code.**
- **Strive to eliminate accidental complexity when possible i.e. complexity that is not inherent to the problem being solved but only introduced by the implementation.**
- **Specifically for a data processing library, think about the data transformation in terms of “pipelines” and design/architect around that**



Connect with me



Talk slides

THANK YOU!